

Wraith: Private Conditional Orders via Trusted Execution and Enshrined Oracles on Flare

Project Wraith

Flare Summer Signal Hackathon — Confidential Compute & Interoperable Asset Products tracks

August 16, 2026 · Deployment: Flare Coston2 (chain 114), contract 0xd5A5322F3D9bB9b2Ee73d006383BB03f61A04eCD

Abstract

Every widely deployed on-chain automation protocol publishes the trigger condition of a standing order in cleartext. A resting stop-loss therefore announces, hours or days in advance, the exact price at which its owner will be forced to sell — and pushing the market into a visible cluster of stops is profitable arithmetic rather than conspiracy. We present *Wraith*, a system in which the condition of a conditional order exists in plaintext in exactly one place: inside a trusted execution environment (TEE) registered with the Flare network. Orders are sealed in the user’s browser under ECIES to the enclave’s public key and stored on-chain as ciphertext beside an escrow; an untrusted, permissionless keeper can prompt evaluation but can neither read nor influence it, because the enclave reads Flare’s enshrined FTSO price feeds itself and verifies cross-chain facts through on-chain FDC attestation checks. When a condition is met, the enclave signs a domain-separated result that the contract verifies against Flare’s TEE machine registry before settling by DEX swap or by FAssets redemption to native XRP. We formalize the trigger calculus for six order kinds — threshold and bracket stops, trailing stops, seed-randomized TWAP schedules, collateral-health escapes, cross-chain payment triggers, and dual-oracle consensus orders — give the verification and replay-protection construction, and analyze what the design hides, what it necessarily reveals, and which trust assumptions remain. The reference implementation is deployed on Coston2 with 279 passing tests across the contract, enclave, keeper, and client.

Keywords: conditional orders; stop-loss hunting; trusted execution environments; Flare; FTSO; FDC; FAssets; ECIES; MEV.

1 Introduction

Public blockchains execute conditional logic transparently, and transparency is precisely the wrong property for a resting order. A limit or stop order is a commitment of the form “when the market reaches T , I will trade size q ”; publishing (T, q) converts the commitment into a target. The practice of driving price into known stop levels — *stop hunting* — predates blockchains, but on-chain automation systems make it strictly easier: the trigger is not inferred from order-book heuristics, it is stored in a public contract variable. Keeper networks such as Gelato and Chainlink Automation, on-chain limit-order books, and DeFi liquidation engines all operate this way, and traders respond rationally by keeping stops off-chain or mental, trading custody and execution risk for privacy.

Wraith removes the advance warning. The order’s condition — threshold, direction, authorized action, and deadline — is encrypted in the owner’s browser to a key that exists only inside a TEE attested and registered by the Flare network. The chain stores ciphertext and escrow; the enclave alone decrypts, evaluates against oracle data it reads or verifies itself, and signs settlement instructions the contract checks against the network’s machine registry. No party other than the enclave — not the keeper, not the contract owner, not the operator of the enclave host — ever observes the condition.

The claim is deliberately narrow. Wraith hides *intent before execution*; it does not hide execution itself. A fired trigger produces an ordinary public transaction, as exposed to execution-moment MEV as any other trade (Section 9). We consider the narrow claim the defensible one: the condition was never public, so it could never be hunted.

Flare is, to our knowledge, the only network on which this construction closes without third-party trust: the price oracle (FTSO) is enshrined in the network and readable from inside the enclave over RPC; cross-chain facts arrive through an enshrined attestation protocol (FDC) whose Merkle proofs are verified *on-chain* before the enclave will consider them; TEE identity is a network-level registry rather than an operator allowlist; and settlement to a non-EVM chain (XRPL) proceeds through FAssets redemption without a bridge. Wraith composes all four.

1.1 Contributions

- A design and deployed implementation for conditional orders whose conditions are never published, using Flare Confidential Compute extensions (Section 3–4).
- A trigger calculus covering six order kinds, including a stateless seed-randomized TWAP whose schedule is unpredictable yet restart-safe, and a dual-oracle consensus trigger with a bounded-deviation circuit breaker (Section 5).
- A settlement verification construction binding enclave results to a specific deployment, chain, and single use, with signer authority delegated to the network’s TEE registry rather than any owner-controlled list (Section 7).
- An explicit privacy and trust analysis, including what the system reveals by construction and which residual assumptions a user accepts (Sections 9–10).

2 Threat Model and Design Goals

We consider four principals: the *owner* who seals an order; the *keeper*, an untrusted and permissionless process that prompts evaluation; the *enclave*, a TEE running the Wraith extension registered under Flare extension id 0x102b7; and the *adversary*, who observes all on-chain state and all keeper traffic, may operate keepers, and may attempt to move one price source.

Goals. (G1) *Condition privacy*: the adversary learns nothing about (*T*, direction, action, deadline) beyond what firing itself reveals. (G2) *Keeper minimization*: a hostile keeper can at worst censor; it cannot forge a trigger, feed a false price, or learn the condition. (G3) *Non-forgable settlement*: the contract settles only on results provably signed inside a registry-listed enclave, at most once each. (G4) *Oracle-manipulation resistance (optional, per order)*: a consensus order fires only when two independent price sources agree the level was crossed. (G5) *Honest failure*: when the enclave cannot answer safely — stale data, expired order, malformed terms — it refuses rather than guesses.

Non-goals. Execution privacy (the settlement transaction is public); liveness against a total keeper strike (mitigated by permissionless ticking); and confidentiality against an adversary who breaks the TEE itself (Section 10).

3 System Architecture

Figure 1 shows the components. The on-chain core is a single contract, `WraithOrders` (782 lines of Solidity), holding per-order escrow, ciphertext, and lifecycle flags. The enclave runs a Go extension inside Flare’s Confidential Compute scaffold; its only inbound surface is the instruction relay, and its private key never leaves the TEE node. The keeper is a stateless Node.js loop; the client is a browser application that performs sealing locally.

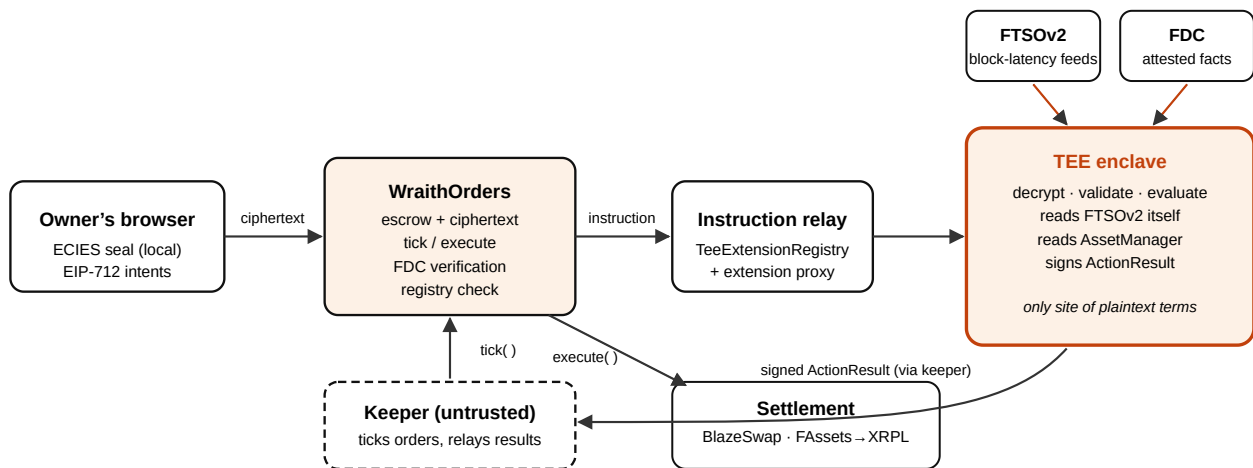


Figure 1. Wraith architecture. Solid boxes are trusted for integrity only; the shaded enclave is the sole confidentiality boundary. The keeper (dashed) is untrusted and replaceable: it forwards bytes it cannot read and relays signatures it cannot forge. Price data enters the enclave directly from FTSoV2; attested facts enter only after on-chain FDC verification.

3.1 Why the enclave reads the oracle itself

An earlier design had the keeper supply the price with the tick. That places the keeper in the integrity path: a lying keeper could fire any order by fabricating an observation. In Wraith the instruction carries no price at all — the enclave fetches `FtsoV2.getFeedById` over RPC from inside the TEE and rejects observations older than 2 minutes. The keeper degenerates to a scheduling hint, which is goal G2. The same reasoning excludes attested facts: the enclave cannot call FDC (a Flare *system* application, Section 6), so proofs are verified by the contract before relay, and the enclave refuses any attestation not marked chain-verified.

4 Order Lifecycle

An order passes through three phases (Figure 2): *seal*, *watch*, *fire*.

Seal. The client serializes the terms τ (Section 5), encrypts them under the enclave public key retrieved from the machine registry (Section 8), and calls `createOrder(ct, token, amount, expiry)`, escrowing the input amount. A gasless variant, `createOrderFor`, accepts an EIP-712 intent signed by the owner covering every economic field — owner, token, amount, expiry, relayer fee, deadline, and the hash of the ciphertext — with a per-signer nonce; the relayer can refuse to submit, but cannot alter (Section 7.2).

Watch. Anyone may call `tick(orderId)`, paying the relay fee. The contract forwards the ciphertext (with public context: order id, escrow remaining, trailing peak, verified attestation if supplied) through `TeeExtensionRegistry.sendInstructions`. Inside the enclave the extension decrypts, validates, gathers observations, and evaluates. A non-firing evaluation produces a no-op result; the condition is not disclosed either way.

Fire. On a firing evaluation the enclave signs an `ActionResult` naming this contract, the order, and the settlement parameters. Any party relays it to `execute()`, which verifies the signature against the registry, consumes the action id, and settles: a swap through a Uniswap-V2-compatible router (BlazeSwap), or a lot-granular FAssets redemption that causes the agent to deliver native XRP to the owner’s XRPL address.

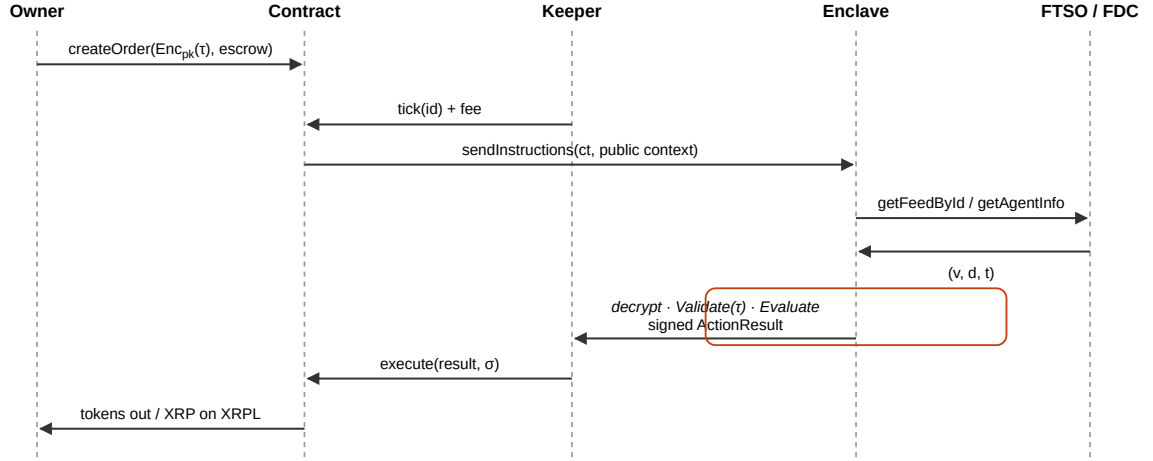


Figure 2. Settlement sequence for a firing tick. The keeper appears twice but contributes no data the enclave or contract depends on: the price observation travels the right-hand edge, and the result it relays is integrity-protected by the enclave signature. The boxed region is the only computation over plaintext terms.

5 Trigger Calculus

All comparisons happen in a single fixed-point domain. FTSOv2 returns an observation (v, d, t) with value v , decimals d , timestamp t ; the true price is $v \cdot 10^{-d}$. The enclave normalizes every source to 18 decimals:

$$P_{18}(v, d) = \begin{cases} v \cdot 10^{18-d}, & d \leq 18 \\ \lfloor v / 10^{d-18} \rfloor, & d > 18 \end{cases} \quad (1)$$

with the exponent bounded, $|18 - d| \leq 60$, so a corrupt feed cannot request an unbounded big-integer allocation. Thresholds are sealed already scaled, $T \equiv T_{18}$. Observations older than $\Delta_{\text{price}} = 2$ min are rejected (goal G5); staleness is measured as $|\text{now} - t|$ since enclave and source clocks may disagree in either direction.

5.1 Threshold and bracket orders

A price order carries direction $\delta \in \{\text{below}, \text{above}\}$ and fires on the inclusive crossing predicate

$$C(P) = (\delta = \text{below} \wedge P \leq T) \vee (\delta = \text{above} \wedge P \geq T) \quad (2)$$

Inclusivity is a product decision made explicit: a stop set *at* the traded price fires, which is what “stop at T ” means to a trader. An optional second threshold T' converts the order into a bracket (one-cancels-other): the opposite side of δ also fires, $C'(P) = C(P) \vee (P \text{ beyond } T' \text{ on the opposite side})$. Sealing-time validation requires T' strictly beyond T — overlapping legs would fire at every price — and the contract’s executed flag settles whichever leg lands first, cancelling the other with no additional mechanism.

5.2 Trailing stops

A trailing stop follows the price up and fires a sealed distance τ (basis points, $0 < \tau < 10^4$) below the running peak. The enclave is stateless across ticks, so the high-water mark H is persisted *on-chain* and ratchets monotonically:

$$H_k = \max(H_{k-1}, P_k), \quad S_k = \frac{H_k \cdot (10^4 - \tau)}{10^4}, \quad \text{fire} \Leftrightarrow H_k > 0 \wedge P_k \leq S_k \quad (3)$$

Publishing H is sound: the peak is a function of public FTSO prices, so it reveals nothing an observer could not compute. The secret is τ alone — without it, knowledge of the peak says nothing about where the order fires. This

is the design’s general pattern: state derived from public inputs may live on-chain; parameters expressing intent may not.

5.3 Stealth TWAP

A TWAP order releases an escrow E in $n \leq 1000$ tranches across a window $[t_s, t_e]$. A public TWAP leaks its own future: equal tranches at equal spacing let an observer front-run every remaining release and infer the total. Wraith derives both the schedule and the sizes from a sealed 32-byte seed σ . Define the deterministic jitter

$$J(\sigma, i, m) = \text{SHA-256}(\sigma \parallel i)[0..8) \bmod m \quad (4)$$

with the chunk index i encoded big-endian in 8 bytes. With slot length $\ell = (t_e - t_s)/n$, tranche i becomes due at boundary

$$b_i = t_s + \ell i + J(\sigma, i, \ell-1), \quad \text{due}(t) = |\{i : t \geq b_i\}|, \quad (5)$$

jitter confined within the slot so the schedule always completes inside the window, and $\text{due}(t) = n$ forced for $t \geq t_e$ so a TWAP always finishes. Progress is inferred from the contract’s remaining escrow R rather than from enclave memory: with per-tranche size $c = \lfloor E/n \rfloor$ and spent $= E - R$, the release at time t is

$$\text{chunk}(t) = \min(E, c \cdot \text{due}(t)) - (E - R), \quad \text{then } \text{chunk} \leftarrow \text{chunk} \cdot \frac{10^4 - J(\sigma, \text{due}+10^3, 3000)}{10^4}, \quad \text{chunk} \leftarrow \min(\text{chunk}, R) \quad (6)$$

The multiplicative shave (up to 30%, applied only before the final tranche and only downward, so the schedule can never overdraw) makes consecutive tranches unequal, denying the observer the division that would recover E . Because everything is recomputed from (σ, E, R, t) , an enclave restart loses no state, cannot double-spend, and cannot stall — a property we call *stateless stealth scheduling*. Figure 3 illustrates a derived schedule.

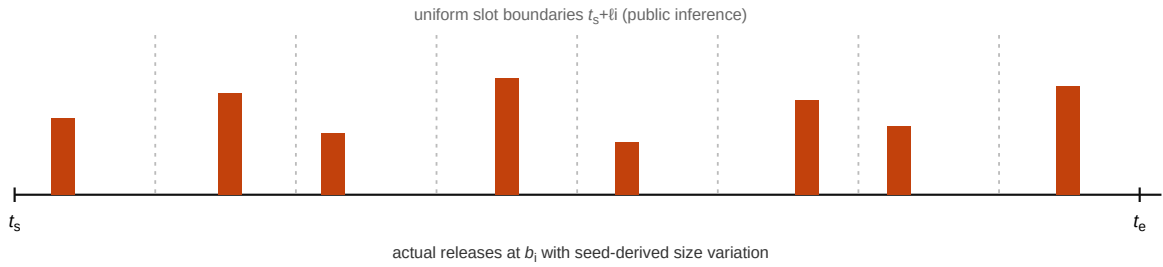


Figure 3. A seed-randomized TWAP schedule ($n = 8$). Dashed lines mark the uniform boundaries an observer would assume; bars mark the actual jittered releases with shaved sizes. Both offsets and sizes are pure functions of the sealed seed, so the schedule is unpredictable to observers yet exactly reproducible by the enclave after any restart.

5.4 FAssets Shield (collateral-health escape)

FXRP holders are exposed to the agent backing their asset. A Shield order watches one agent’s `getAgentInfo` reading — status and vault/pool collateral ratios in basis points — and escapes when either the status leaves NORMAL (CCB, liquidation, destroying) or either ratio falls to the sealed floor κ :

$$\text{fire} \Leftrightarrow \text{status} \neq \text{NORMAL} \vee \text{CR}_{\text{vault}} \leq \kappa \vee \text{CR}_{\text{pool}} \leq \kappa \quad (7)$$

The status check is deliberately independent of the ratios: an agent in liquidation may report ratios that no longer describe its risk. Because κ is sealed, no adversary can position against the exit level of a large holder — the exact scenario a public escape threshold invites.

5.5 Cross-chain triggers

A cross-chain order fires when an FDC-attested XRPL payment to a watched address reaches a sealed amount: $\text{fire} \Leftrightarrow A_{18} \geq T$, where A_{18} is the attested amount, normalized. Before the threshold is consulted the enclave enforces three guards, each closing a distinct keeper attack: the attestation must be marked *chain-verified* (an unverified attestation is the keeper's claim, not FDC's finding); the attested source must equal the sealed watched address (a payment to someone else must not fire this order); and the observation must be fresh, $|\text{now} - t_{\text{att}}| \leq 30 \text{ min}$, sized for FDC's 90–180 s rounds plus relay. The watched address travels as the FDC standard address hash on both sides, so it is not published either.

5.6 Consensus orders (dual oracle)

Privacy stops an adversary aiming at a trigger it cannot see; it does not stop walking a single price feed until *something* fires. A consensus order requires two independent sources — the FTSO feed and an FDC-attested off-chain price — to *both* cross the threshold, and refuses to act when they disagree beyond a sealed tolerance δ_{max} (basis points):

$$\Delta = \frac{|P_{\text{ftso}} - P_{\text{att}}| \cdot 10^4}{P_{\text{ftso}}} \quad \text{fire} \Leftrightarrow C(P_{\text{ftso}}) \wedge C(P_{\text{att}}) \wedge \Delta \leq \delta_{\text{max}} \quad (8)$$

The conjunction is the mechanism: forcing two independent sources through a level at the same moment is a categorically harder attack than nudging one. The deviation guard is a circuit breaker rather than a trigger — a wide gap between two honest sources means one is wrong, and acting on either is worse than waiting. Validation rejects $\delta_{\text{max}} \geq 10^4$, which would accept any pair of readings and collapse the two oracles back into one.

5.7 Validation before evaluation

Every kind is validated once, at decrypt, against structural coherence — brackets must not overlap, trails must lie in (0, 100%), TWAP windows must move forward, settlement legs must be complete — so a malformed order is rejected loudly rather than silently never firing. Evaluation returns an *error*, not a false decision, whenever it cannot answer safely (goal G5); the contract will not settle a failed result.

6 Attested Facts: the FDC Path

The TEE-based FDC is a Flare system application: extension ids below 0x10000 are reserved and there is no developer interface callable from inside a third-party enclave. Wraith therefore inverts the flow. The keeper requests an attestation at the verifier, submits it to FdcHub with the round fee, waits for finalization (typically 90–180 s), and fetches the Merkle proof from the Data Availability layer; the voting round is recovered from the carrying block's timestamp as

$$r = \frac{t_{\text{block}} - t_0}{\Delta_{\text{epoch}}} \quad (9)$$

where t_0 is the first voting-round start and Δ_{epoch} the epoch duration. The contract's `tickAttested` (XRPL Payment type) and `tickAttestedWeb2` (Web2Json type) verify the proof against the finalized round *on-chain* and only then relay the reading into the instruction. For Web2 sources the attestation post-processes the API response with a jq program into the ABI tuple (`source`, `value`, `decimals`, `timestamp`) — the reading carries its own scale, because FDC's jq subset lacks `floor` and string-truncating a float already rendered in scientific notation fails silently by orders of magnitude; the contract performs the widening of equation (1). One attestation serves every order ticked in the same window, since re-requesting per order buys no additional assurance about the same fact.

The division of labor is exact: *the chain verifies provenance, the enclave applies the secret*. The observed datum (a payment amount, an API price) becomes public through the attestation; the threshold it is compared against never does. Knowing that 100 XRP landed at some address says nothing about which orders, if any, care.

7 On-Chain Verification

7.1 Settlement authentication

The enclave signs a domain-separated digest over the result. With d the ABI-encoded settlement data, a the action id, g the submission tag, and s the status code:

$$h_{\text{res}} = \text{keccak256}(\text{keccak256}(d) \parallel a \parallel \text{keccak256}(g) \parallel s) \quad (10)$$

$$h_{\text{pay}} = \text{keccak256}(\text{abi.encode}(\text{"TEE_ACTION_RESULT"}, \text{chainid}, h_{\text{res}})), \quad \sigma = \text{Sig}_{\text{tee}}(\text{EIP-191}(h_{\text{pay}})) \quad (11)$$

`execute()` recovers the signer by `erecover` and accepts only if the signer appears in `TeeMachineRegistry.getActiveTeeMachines(extensionId)` — the network’s registry, not an owner-controlled allowlist. The contract owner cannot authorize a signer of their choosing (a test asserts exactly this), and retiring a machine in the registry revokes its settlement authority immediately with no action from the deployer. Table 1 maps each check to the attack it forecloses.

Table 1. Checks performed by `execute()` and the attack each one forecloses. All five must hold.

Check	Prevents
Recovered signer $\in$ registry machines for extension 0x102b7	Forged or operator-injected results
Action id a not previously consumed	Replaying one signed result to settle twice
Result names <code>address(this)</code>	Cross-deployment replay
chainid inside h_{pay}	Cross-chain replay
Status $s = 1$; order live (not executed / cancelled / expired)	Settling a failed evaluation or a dead order

7.2 Gasless creation

The EIP-712 intent for `createOrderFor` covers (owner, token, amount, expiry, relayerFee, deadline, nonce, $\text{keccak256}(\text{ct})$) under the contract’s domain separator. Because the ciphertext hash is inside the signature, a relayer cannot substitute different sealed terms; because the fee is, it cannot raise its own price; the per-signer nonce makes each intent single-use. A relayer’s only power is refusal — and any other relayer can take the same intent, the same permissionless-substitution property the keeper design relies on.

8 Cryptographic Construction

Sealing is ECIES over `secp256k1` in the go-ethereum construction, chosen because it is what the TEE node’s decryption endpoint implements: an ephemeral key exchange followed by NIST Concat-KDF (SHA-256), AES-128-CTR encryption, and an HMAC-SHA-256 tag over the ciphertext. For enclave public key pk and plaintext terms m :

$$k \leftarrow_{\mathbb{R}} \mathbb{Z}_q, \quad R = kG, \quad Z = (k \cdot pk)_x, \quad (k_E \parallel k_M) = \text{KDF}(Z) \quad (12)$$

$$\text{ct} = 0x04 \parallel R \parallel \text{IV} \parallel \text{AES-CTR}_{k_E}(m) \parallel \text{HMAC}_{k_M}(\cdot) \quad (13)$$

The fixed overhead is 113 bytes (65-byte uncompressed ephemeral point, 16-byte IV, 32-byte MAC); a representative sealed order on Coston2 is 785 bytes of calldata. Encryption happens in the browser; the enclave key is fetched from the machine registry and cross-checkable by anyone against the chain. Decryption happens only at the TEE node’s local /decrypt endpoint — the private key never crosses the enclave boundary, and the extension only ever handles plaintext inside it. The client library deliberately uses the Concat-KDF/CTR/HMAC suite rather than the HKDF/AES-GCM suite of other ECIES libraries; the two are wire-incompatible, and an order sealed under the wrong one is permanently unopenable.

9 Privacy Analysis

Table 2 states the information available to each principal. The system’s privacy claim reduces to the last column: only the enclave column contains the condition.

Table 2. Knowledge matrix. “Fired” becomes public to all parties at settlement, which is inherent: settlement is a public transaction.

Datum	Public / adversary	Keeper	Contract owner	Enclave
Order existence, owner, escrow token & amount, expiry	yes	yes	yes	yes
Ciphertext bytes	yes	yes	yes	yes
Trigger threshold, direction, action, bracket, trail, seed, tolerance	—	—	—	yes
Trailing peak H (derived from public prices)	yes	yes	yes	yes
Whether a given tick fired	at settlement	on relay	at settlement	yes
Distance between price and threshold	—	—	—	yes

What firing reveals. Settlement discloses that *some* condition of this order was satisfied at that moment, which bounds the threshold to one side of the then-current price. This is unavoidable for any system that settles on-chain, and it is disclosure *after* the fact — the hunting attack requires the level in advance. The TWAP construction extends the argument to repeated partial settlements: equation (6)’s size shave and equation (5)’s boundary jitter deny the observer both the count and the cadence, so watching k tranches land does not predict the $(k+1)$ -th.

Side channels. Tick timing is public; keepers tick on a fixed cadence for all live orders, so cadence carries no per-order information. A keeper that relays a firing result learns the fire before the chain does, but never the threshold or margin. Ciphertext length is padded only by the ABI encoding; length variation across order kinds is a real, low-bandwidth channel and is noted as such.

What is not claimed. Execution privacy. A fired swap enters a public mempool with ordinary MEV exposure. Claims that a TEE makes trading “MEV-proof” conflate intent privacy with execution privacy; Wraith provides the first and explicitly not the second.

10 Trust Assumptions and Limitations

Stated plainly, in decreasing order of weight.

1. **TEE platform trust.** A TEE relocates trust rather than eliminating it: the silicon vendor’s attestation, the enclave’s isolation against side channels, and Flare’s registration of the machine are all assumed. This surface is smaller and more auditable than “trust the operator,” but it is not zero.

2. **Simulated TEE in the current deployment.** The Coston2 deployment runs with `SIMULATED_TEE=true`: the signature path, instruction lifecycle, registry checks, and contract logic are all real; the hardware attestation is not. Production requires Confidential Space hardware.
3. **Durable ciphertext exposure.** Ciphertext stored on-chain can be attacked indefinitely; Flare’s own guidance recommends off-chain secret delivery for production. For a stop-loss with a horizon of days the trade is acceptable; for long-lived secrets it is not. An off-chain delivery channel is the largest open item.
4. **Volatile enclave identity.** The TEE keypair is generated at boot and there is no sealed storage. A restart mints a new identity: pending orders sealed to the old key become permanently undecryptable and must be re-sealed. The on-chain ciphertext being canonical makes this a liveness loss, never a safety loss.
5. **Keeper censorship.** A keeper can decline to tick. Ticking is permissionless, so any party — including the owner — can substitute; the residual risk is a total strike of all willing keepers.
6. **Consensus-order source independence.** FDC proves an API *said* something, not that it was right. A consensus order assumes the attested Web2 source is not controlled by the same adversary as the FTSO feed.
7. **Testnet status.** Flare Confidential Compute is pre-release; Wraith targets Coston2 and should hold no real funds.

11 Implementation and Evaluation

The reference implementation comprises the 782-line `WraithOrders` contract (Foundry), a dependency-free Go trigger package plus enclave handler inside the FCC extension scaffold, a stateless Node.js keeper, and a Next.js client performing ECIES sealing locally. 279 tests pass across the four components; the contract suite covers signature forgery, action-id replay, cross-deployment replay, owner-cannot-allowlist, bracket-leg validation, and FDC decode round-trips, and the trigger package is pure so every predicate in Section 5 is tested directly against boundary cases (threshold equality, expired orders, stale observations, hostile decimals, overlapping brackets, degenerate trails and windows).

Table 3. Deployment coordinates (Coston2 testnet).

Component	Value
Network	Flare Coston2, chain id 114
WraithOrders	0xd5A5322F3D9bB9b2Ee73d006383BB03f61A04eCD
FCC extension id	0x102b7 (66231)
Price feeds	FTSOv2 block-latency feeds, read in-enclave
Attestation types	FDC Payment (XRPL), Web2Json (second oracle)
Settlement	BlazeSwap router; FAssets FXRP redemption to native XRP

12 Related Work

Keeper networks. Gelato and Chainlink Automation execute user-registered conditions faithfully but publicly; the condition is calldata. Wraith is complementary: it keeps the keeper role and removes its knowledge. **Intent systems.** CoW Protocol and other batch-auction designs hide orders pre-settlement from the public but disclose them to solvers; the trust boundary moves to the solver set, and standing conditional intent is out of scope. **Commit–reveal** hides parameters until execution but requires the owner (or a delegate who learns the secret) to come online and reveal at exactly the right moment — reintroducing the availability problem conditional orders exist to solve. **TEE-based chains and coprocessors** (e.g. secret-contract chains and SUAVE-style proposals) offer general confidential computation; Wraith is instead a minimal composition over an existing L1’s enshrined

oracles, inheriting FTSO/FDC security rather than bootstrapping a separate network. **Dark pools** solve the disclosure problem off-chain by trusting the venue; Wraith’s venue is a registry-attested enclave with on-chain verification of every settlement.

13 Future Work

Hardware attestation on Confidential Space; off-chain ciphertext delivery with an on-chain commitment (removing limitation 3); sealed-storage or threshold re-encryption to survive enclave restarts (limitation 4); length-padding the ciphertext to a uniform size; multi-enclave evaluation with k -of- n result agreement, which would soften the single-TEE trust assumption the same way the consensus order softens single-oracle trust; and mainnet deployment once FCC exits pre-release.

14 Conclusion

A conditional order is a secret with a settlement attached. Wraith keeps the secret in the one place it must exist to be enforced — a network-registered enclave — and makes every other participant either blind (the keeper), bound (the contract), or both. The composition is only possible because Flare enshrines its oracles: prices the enclave can read without trusting a relayer, attestations the chain can verify before the enclave consumes them, and a TEE registry the contract can consult instead of an owner’s allowlist. The result is a standing order that never announces itself — and a privacy claim narrow enough to survive questioning.

References

1. Flare Networks. *Flare Confidential Compute: Overview and Extension Development Guides*. dev.flare.network/fcc, 2026.
2. Flare Networks. *FTSOv2: Block-Latency Feeds*. dev.flare.network/ftso, 2026.
3. Flare Networks. *Flare Data Connector: Attestation Types and Verification*. dev.flare.network/fdc, 2026.
4. Flare Networks. *FAssets: Trustless Representation of Non-Smart-Contract Assets*. dev.flare.network/fassets, 2026.
5. Daian, P. et al. *Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges*. IEEE S&P, 2020.
6. Eskandari, S., Moosavi, S., Clark, J. *SoK: Transparent Dishonesty — Front-Running Attacks on Blockchain*. FC WTSC, 2019.
7. Certicom Research. *SEC 1: Elliptic Curve Cryptography (ECIES)*, v2.0, 2009.
8. NIST. *SP 800-56A: Recommendation for Pair-Wise Key-Establishment Schemes (Concat-KDF)*, rev. 3, 2018.
9. Ethereum. *EIP-712: Typed Structured Data Hashing and Signing; EIP-191: Signed Data Standard*.
10. Costan, V., Devadas, S. *Intel SGX Explained*. IACR ePrint 2016/086. (TEE background and side-channel caveats.)
11. Flashbots. *SUAVE: Single Unifying Auction for Value Expression*. 2023. (TEE-based confidential order flow.)
12. CoW Protocol. *Batch Auctions and Solver Competition*. docs.cow.fi, 2024.
13. Project Wraith. *Source, trust assumptions, and deployment proofs*. github.com/LSUDOKO/Wraith, 2026.

Wraith is hackathon software deployed on the Coston2 testnet. Flare Confidential Compute is pre-release; nothing in this paper is an invitation to place real funds behind the current deployment. All formulas correspond line-for-line to the audited evaluation code in `extension/internal/trigger` and `contracts/src/WraithOrders.sol` at the commit tagged for submission.